

Technical Assignment — Junior Software Developer (Frontend + [Node.js](#))

Position: Junior Software Developer

Stack: React / Angular + Node.js

Time Limit: 1 Day (take-home)

Submission Deadline:

Overview

Build a **Task Manager** — a simple full-stack web application that lets users create, view, update, and delete tasks. This assignment evaluates your ability to work across the frontend and backend, write clean code, and follow basic software engineering practices.

Requirements

Backend (Node.js)

Build a REST API using **Express.js** (or a framework of your choice) with the following endpoints:

Method	Endpoint	Description
GET	/api/tasks	Fetch all tasks (support filtering by status)
GET	/api/tasks/:id	Fetch a single task by ID
POST	/api/tasks	Create a new task
PUT	/api/tasks/:id	Update an existing task
DELETE	/api/tasks/:id	Delete a task

Task Object Structure:

Json

```
{
  "id": "string (auto-generated)",
  "title": "string (required, max 100 chars)",
  "description": "string (optional, max 500 chars)",
  "status": "pending | in-progress | completed",
  "priority": "low | medium | high",
  "createdAt": "ISO date string",
  "updatedAt": "ISO date string"
}
```

Backend expectations:

- Input validation with meaningful error messages and appropriate HTTP status codes
- In-memory storage is acceptable (no database required, but using one is a bonus)
- Proper project structure (routes, controllers, etc. separated logically)
- At least 3 unit tests for any one module (use Jest, Mocha, or a testing library of your choice)

Frontend (React or Angular)

Build a single-page UI that connects to your API and provides the following:

- A form to create a new task (with validation)
- A list/table view showing all tasks with their title, status, and priority
- Ability to edit a task inline or via a modal/separate view
- Ability to delete a task with a confirmation prompt
- Filter tasks by status (pending, in-progress, completed)
- Basic responsive layout (should be usable on mobile screens)

Frontend expectations:

- Component-based architecture with reusable components
- Clean state management (React hooks / Angular services)
- Loading and error states handled in the UI
- Minimal but intentional styling (plain CSS, Tailwind, or a component library — your choice)

Submission Guidelines

1. Push your code to a **public GitHub repository** (or share a private repo with the email/username provided).
2. Include a **README.md** in the root with:
 - A brief description of the project and any design decisions you made
 - Steps to install dependencies and run the app locally
 - Steps to run tests
 - Any known limitations or things you'd improve with more time
1. Ensure the project runs successfully with the steps documented in your README.
2. Reply to the assignment email with the repository link before the deadline.